FootballVision: Player, Ball and Team Tracker

A start-to-finish project guide · Prepared with Claude · 25 October 2026

1. The aim

Build a computer-vision system that takes a football (soccer) match clip and returns the same clip annotated with:

- every player, goalkeeper and referee boxed and given an ID that stays the same across frames
- the ball located frame by frame, with a smooth trail
- players split into two teams by jersey colour
- simple match statistics, starting with a possession estimate

You will get there by fine-tuning a pretrained object detector (Ultralytics YOLO26) on a public football dataset, then building tracking, team assignment and analytics on top of it, and finishing with a small web demo.

The one-sentence version for your portfolio: I fine-tuned a detector on football footage, measured how much it beat the off-the-shelf model, built tracking and team assignment on top, and shipped a demo.

Why this project is worth your time

- Fine-tuning earns its place here. A stock COCO model knows ‘person’ and ‘sports ball’ but has no idea what a ‘referee’ or ‘goalkeeper’ is, and it struggles with a tiny, fast, motion-blurred ball.
- You get a clear before-and-after number, which most beginner projects lack.
- The output is a video, so it is easy to show and share.
- It uses the same building blocks as real sports-analytics tools.

2. How the finished system works

```
match_clip.mp4
   |
   ▼
[1 Detector]        fine-tuned YOLO26 → boxes for player, goalkeeper,
referee, ball
   |
   ▼
[2 Tracker]         ByteTrack → a persistent ID for each person across frames
   |
   ▼
[3 Ball cleanup]    keep the best ball box per frame, reject jumps, fill
short gaps
   |
   ▼
[4 Team assignment] jersey colour → KMeans → team A / team B
   |
   ▼
[5 Analytics]       possession estimate (stretch: top-down pitch view)
   |
   ▼
[6 Output]          annotated .mp4 + tracks.csv + summary stats + Gradio demo
```

What each stage does

1. **Detector.** A neural network that looks at one frame and returns boxes, a class and a confidence score for each. Starting from pretrained weights means it already understands edges, shapes and people; fine-tuning teaches it your four classes.
2. **Tracker.** Detection works frame by frame and has no memory. The tracker links boxes across frames using predicted motion, so ‘player #7’ stays #7. It is built into Ultralytics.
3. **Ball cleanup.** There is only one ball, but the detector may report several candidates or none. You keep the most confident one, throw away impossible jumps, and fill short gaps.
4. **Team assignment.** Crop each player’s torso, take its median colour in LAB colour space, cluster all players into two groups with KMeans, then let each track vote on its team.
5. **Analytics.** Use the ball and player positions to estimate which team has the ball.
6. **Output.** Draw everything on the video, save the raw data, and wrap it in an app.

Inputs: a video clip (10–30 s for development). **Outputs:** annotated video, a per-frame CSV (frame, ID, class, box, team), and a short stats summary.

3. Scope

Level	What's included
Must have (MVP)	Fine-tuned detector evaluated against the off-the-shelf baseline; tracking with IDs on a 30 s clip; ball cleanup; README with results and a demo video
Should have	Team assignment by jersey colour; possession estimate; Gradio demo on Hugging Face Spaces
Stretch	Pitch keypoints + homography for a top-down radar view, distance covered, speed and real heatmaps; a second dataset; simple event detection such as passes
Out of scope	Live streaming, identifying named players or reading shirt numbers, offside calls, commercial use

4. Success criteria

Set your actual targets after you have a baseline. These are sensible starting goals, not promises.

Measure	How you check it	Starting goal
Player detection quality	mAP50 on validation split and on your own 30-frame test set	High (aim for 0.90 or above on validation)
Ball detection	% of frames in your held-out clip with a correct ball box, before and after cleanup	Clearly better than the COCO baseline
Improvement over baseline	Same clip, COCO model vs your model (ball and person only)	A table showing the gap
Tracking	Unique player IDs vs players actually visible; ID switches counted by eye in three 10 s segments	Few switches; IDs mostly stable

Measure	How you check it	Starting goal
Team assignment	Check 50 random player crops by eye	90% correct or better
Speed	Frames per second on a Colab T4	Report it honestly, whatever it is

5. Tools, accounts and setup

Accounts (all free): Google (for Colab and Drive), Roboflow, GitHub, Hugging Face.

Libraries: `ultralytics` (training and tracking), `roboflow` (dataset download), `opencv-python`, `pandas`, `scikit-learn`, `gradio`. The `supervision` library is optional but handy for drawing boxes, trails and zones.

Hardware: Colab free tier with a T4 GPU. Colab wipes its disk when the session ends, so copy every `best.pt` and result to Google Drive straight after each run.

Suggested repo layout

```
footballvision/
├── README.md
├── notes.md              # observations, failures, ideas
├── experiments.md        # the experiment log (section 8)
├── notebooks/            # Colab notebooks, one per phase
├── src/
│   ├── run.py            # the full pipeline as a command-line tool
│   ├── ball.py
│   ├── teams.py
│   └── stats.py
├── app.py                # Gradio demo
├── assets/               # demo GIFs, result charts, failure gallery
└── requirements.txt
```

6. Phase-by-phase plan

Times are rough estimates for a beginner working part-time. Each phase ends with a ‘done when’ check, so you always know whether to move on.

Phase 0 · Setup and baseline (about 3 hours)

Goal: see what the off-the-shelf model gets wrong, so your improvement has something to be measured against.

1. Create the GitHub repo and folders above.
2. Open Colab, choose Runtime → Change runtime type → T4 GPU, then run `pip install ultralytics roboflow supervision scikit-learn opencv-python pandas gradio`.
3. Pick two broadcast-style clips of 20–30 s each. **Clip A** is for demos. **Clip B is your held-out test clip and must never be used for training.** Use footage you have the right to use (your own recording, or a clip whose licence allows it).
4. Run the COCO-pretrained model and save the result:

```
from ultralytics import YOLO
base = YOLO('yolo26n.pt')                          # COCO-pretrained
base.predict('clipB.mp4', save=True, classes=[0, 32])   # 0 = person, 32 =
sports ball
```

5. Watch the output and write 5 or more failures in `notes.md` (missed ball, referee labelled as person, and so on).

Done when: the baseline video is saved and your failure notes are written.

Phase 1 · Data (about 3 hours)

Goal: understand the dataset before you train on it, and build an honest test set.

1. Use the `football` project by processing-workspace on Roboflow Universe (`universe.roboflow.com/processing-workspace/football-l9zew`). Version 6 had 753 images with ball, player, referee and goalkeeper classes when I checked. Pages and versions change, so if the download fails, open the page and copy the snippet from its Download button.
2. Download in YOLO format (get your API key from Roboflow Settings):

```
from roboflow import Roboflow
rf = Roboflow(api_key='YOUR_KEY')
project = rf.workspace('processing-workspace').project('football-l9zew')
dataset = project.version(6).download('yolov8')   # generic YOLO export
format
```

3. Open `data.yaml` and confirm the exact class names (you will need them later, for example `ball` vs `Ball`).

4. Count instances per class:

```
from pathlib import Path
from collections import Counter
root = Path(dataset.location)
counts = Counter()
for f in (root / 'train' / 'labels').glob('*.txt'):
    for line in f.read_text().splitlines():
        counts[int(line.split()[0])] += 1
print(counts)
```

5. Look at 30 random annotated images. Note label quality, camera angles, and how many balls are visible. Check the dataset's licence on its page and write it in your README.
6. **Build your own test set:** take about 30 frames from Clip B (one per second), upload them to Roboflow, label the four classes yourself, and export. This takes an hour or so, and it is the most trustworthy number in your whole project, because frames from the same video often appear in both train and validation in public datasets, which inflates their scores.
7. Back up the downloaded dataset zip to Drive in case the project is removed or changed.

Done when: you have a class-count table in your notes and a 30-frame test set exported.

Phase 2 · Train the detector (about 6 hours)

Goal: a fine-tuned model plus a small set of controlled experiments.

```
from ultralytics import YOLO
model = YOLO('yolo26n.pt')          # smallest model: fast first run
model.train(data=f'{dataset.location}/data.yaml', epochs=60, imgsz=960,
            batch=16, patience=15, project='/content/drive/MyDrive/fv_runs',
name='n_960')
```

- The ball is tiny, which is why `imgsz=960` rather than the default 640. If you run out of GPU memory, lower `batch` to 8.
- Save runs to Drive (as above) so a Colab disconnect doesn't cost you the work.
- After the first run works, change **one thing at a time** and log it (section 8): model size (`yolo26n` → `yolo26s`), image size (960 → 1280), epochs, augmentation settings.

Done when: at least four runs are logged and the best `best.pt` is on Drive.

Phase 3 · Evaluate and analyse errors (about 4 hours)

Goal: know exactly where the model is strong and weak.

1. Validate on the dataset's validation split and on your own 30-frame test set (make a small `data.yaml` that points to your frames):

```
best = YOLO('/content/drive/MyDrive/fv_runs/n_960/weights/best.pt')
m = best.val(data='my_test/data.yaml')
print('mAP50:', m.box.map50, ' mAP50-95:', m.box.map)
```

2. Read the per-class scores and the confusion matrix in the run folder. Expect the ball to be the weakest class.
3. Build a **failure gallery**: 20 worst frames, grouped by cause (ball missed, goalkeeper vs player confusion, referee vs player, crowded penalty box, motion blur).
4. Compare baseline vs fine-tuned on Clip B for the ball and for people: the percentage of frames with a ball box above confidence 0.25. The COCO model has no referee or goalkeeper class, so leave those out of the comparison.
5. If the ball is poor, try in this order: bigger image size, a larger model, slicing inference (the SAHI technique), a lower confidence threshold for the ball only, then more ball-rich training data.

Done when: your repo has a results table (baseline vs fine-tuned), the confusion matrix and the failure gallery.

Phase 4 · Tracking (about 5 hours)

Goal: a stable ID for every person, saved as data.

```
import pandas as pd
rows = []
stream = best.track(source='clipB.mp4', tracker='bytetrack.yaml',
                    persist=True, stream=True, conf=0.25)
for frame_idx, r in enumerate(stream):
    ids = r.boxes.id.int().tolist() if r.boxes.id is not None else [-1] *
len(r.boxes)
    for box, c, conf, tid in zip(r.boxes.xyxy.tolist(),
r.boxes.cls.int().tolist(),
                                 r.boxes.conf.tolist(), ids):
        rows.append(dict(frame=frame_idx, id=tid, cls=r.names[c], conf=conf,
                         x1=box[0], y1=box[1], x2=box[2], y2=box[3]))
tracks = pd.DataFrame(rows)
n_frames = frame_idx + 1
tracks.to_csv('tracks.csv', index=False)
```

1. Count unique player IDs and compare with how many players are actually visible. A big excess means tracks are breaking and restarting.
2. Watch three 10 s segments and count ID switches by eye.
3. Try `botsort.yaml` instead of `bytetrack.yaml` and compare.
4. Fix class flicker (a player track briefly labelled goalkeeper): give each track ID its majority class across all its frames.
5. Camera cuts and zooms break tracking. For development, use clips without cuts.

Done when: `tracks.csv` and an annotated video exist, and your ID-switch count is written down.

Phase 5 · Ball cleanup (about 3 hours)

Goal: one believable ball position per frame, with gaps handled honestly.

```
ball = (tracks[tracks.cls == 'ball']       # check the exact class name in
data.yaml
        .sort_values('conf', ascending=False)
        .drop_duplicates('frame')           # keep the most confident ball
per frame
        .set_index('frame')
        .reindex(range(n_frames)))          # missing frames become NaN
cols = ['x1', 'y1', 'x2', 'y2']
ball[cols] = ball[cols].interpolate(limit=10, limit_area='inside')
```

- **Reject jumps:** if the ball centre moves further between consecutive frames than is physically plausible, drop that detection. Pick the threshold by looking at your own clip.
- **Fill only short gaps:** `limit=10` fills up to about 10 frames. It also fills the first 10 frames of a longer gap, so blank out any gap longer than that afterwards. A made-up path is worse than an honest hole.
- Draw a short fading trail behind the ball in the output video.

Done when: you can report ball detection rate before and after cleanup on Clip B.

Phase 6 · Team assignment (about 4 hours)

Goal: colour each player by team.

```
import cv2, numpy as np
from sklearn.cluster import KMeans

def jersey_color(frame, box):
    x1, y1, x2, y2 = map(int, box)
    h, w = y2 - y1, x2 - x1
    crop = frame[y1 + int(.2 * h): y1 + int(.5 * h), x1 + int(.25 * w): x2 -
int(.25 * w)]
    if crop.size == 0:
        return None
    lab = cv2.cvtColor(crop, cv2.COLOR_BGR2LAB)      # torso only: skips
head, shorts, background
    return np.median(lab.reshape(-1, 3), axis=0)

# 1) collect colours of 'player' boxes from the first ~100 frames
# 2) fit once:   kmeans = KMeans(n_clusters=2,
n_init=10).fit(np.array(colors))
# 3) per player, per frame:  team = kmeans.predict([color])[0]
# 4) per track ID: final team = majority vote over all its frames
```

- Exclude referees (by class) from the clustering and draw them in a third colour.
- Goalkeepers wear different kits, so in the MVP give them their own colour. A simple later fix is to assign them to the team whose players are nearest on average.
- Check 50 random player crops by eye and record the accuracy.
- Common failures: similar kit colours, strong shadows, a very zoomed camera. Fixes: tune the crop area, drop green pixels (the pitch) before taking the median, or fit the clusters on more frames.

Done when: the output video shows two team colours and your 50-crop accuracy is written down.

Phase 7 · Possession estimate (about 3 hours)

Goal: a simple, honest statistic.

- For each frame with a valid ball, find the player whose foot point (bottom centre of their box) is closest to the ball centre.
- Count that frame for the player's team only if the distance is below a threshold, for example a fraction of the player's box height. Tune it by eye.
- Smooth the result with a short majority vote (about 5 frames) so possession doesn't flicker.
- Report possession as the share of valid frames per team, and **label it clearly as a 'nearest-player estimate'**, not official possession.
- Do **not** draw pixel-space heatmaps or compute speeds from a panning broadcast camera. The camera moves, so pixel positions don't map to pitch positions. Real heatmaps and distances need Phase 10.

Done when: the output shows a live possession bar and the summary prints per-team percentages.

Phase 8 · Package and demo (about 5 hours)

Goal: anyone can run it.

1. Move the pipeline into `src/run.py` with a command line: `python src/run.py --video in.mp4 --weights best.pt --out out.mp4`. It should write the annotated video, `tracks.csv` and a stats summary.
2. Build a Gradio app (`app.py`):

```
import gradio as gr
from src.run import run_pipeline          # returns (output_video_path,
stats_dict)

def process(video_path):
    return run_pipeline(video_path)

gr.Interface(process, gr.Video(), [gr.Video(), gr.JSON()],
             title='FootballVision').launch()
```

3. Deploy to Hugging Face Spaces. The free tier is CPU-only unless you choose GPU hardware, so cap uploads at about 10 s, downscale to 720p and process every second frame. Also record a pre-rendered demo video so visitors can see good results immediately.
4. Pin versions in `requirements.txt`.

Done when: a stranger can open a link, upload a short clip and get an annotated result.

Phase 9 · Write-up and portfolio (about 4 hours)

Goal: make the work legible to other people.

Your README should contain, in this order:

1. A demo GIF at the top.
2. One-paragraph problem statement and the pipeline diagram.
3. Results table: baseline vs fine-tuned, per class, plus your own test set.
4. The experiment log (what you tried and what helped).
5. Failure gallery and an honest limitations section.
6. How to run it.
7. Data, model and footage licences and credits.

Then write a short post (LinkedIn or a blog) about the one surprising thing you learned, for example why the ball was so hard.

Done when: a recruiter could understand the project in 60 seconds from the README alone.

Phase 10 · Stretch: top-down pitch view (about 8 hours)

Goal: real-world positions, which unlock heatmaps, distance and speed.

1. Find a football pitch-keypoints dataset on Roboflow Universe (search ‘football field keypoints’). Check its licence and how many keypoints it defines.
2. Fine-tune a YOLO pose model on it to detect pitch landmarks such as line intersections and penalty-box corners.
3. Each frame, match the detected landmarks to their known positions on a standard 105 × 68 m pitch template and compute a homography with `cv2.findHomography` (needs at least four well-spread, non-collinear points). Smooth it over time so the view doesn’t jitter.

4. Project every player's foot point through the homography onto the template and draw a radar mini-map.
5. From pitch coordinates you can now compute distance covered, top speed and genuine heatmaps.

Done when: the output video has a radar view and you can print distance per player for a clip.

7. Timeline

About 40 hours for the core project (Phases 0–9) and about 8 more for the stretch. At roughly 10 hours a week:

Week	Phases	Milestone
1	0, 1, 2	Baseline recorded, data understood, first model trained
2	3, 4, 5	Results table, tracking, clean ball path
3	6, 7	Team colours and possession estimate
4	8, 9	Demo live, README finished (**MVP complete**)
5	10	Optional: pitch radar view and real stats

If you fall behind, protect this order: detector → tracking → README. Teams, possession and the pitch view are bonuses.

8. Experiment log template

Keep this in `experiments.md` and add a row after every training run.

Run	Model	imgsz	Epochs	Player mAP50	Ball mAP50	Ball detection rate (Clip B)	Notes
baseline	yolo26n COCO	640	–	–	–		No referee / goalkeeper classes
n_960	yolo26n	960	60				First fine-tune
s_960	yolo26s	960	60				Does a bigger model help the ball?

Run	Model	imgsz	Epochs	Player mAP50	Ball mAP50	Ball detection rate (Clip B)	Notes
n_1280	yolo26n	1280	60				Does resolution help the ball?

9. Risks and fixes

Problem	Likely cause	What to try
Colab disconnects mid-training	Free-tier limits	Save runs to Drive; keep `patience` set; resume from `last.pt`
Out-of-memory error	Image size or batch too large	Lower `batch`, then `imgsz`, or use a smaller model
Great validation score, poor results on Clip B	Near-duplicate frames across train and validation	Trust your own test set; don't report the public validation number alone
Ball keeps disappearing	Tiny, blurred, hidden by players	Bigger `imgsz`, slicing inference, gap filling, more ball data
Many more IDs than players	Occlusion, camera cuts, low confidence	Try `botsort.yaml`, tune the tracker config, use cut-free clips
Teams assigned wrongly	Similar kits, shadows, crop includes pitch	Better torso crop, remove green pixels, fit on more frames
Dataset page changes or disappears	Roboflow projects are user-owned	Keep a zip backup on Drive
Spaces demo is too slow	CPU hardware	Shorter clips, lower resolution, skip frames, or show a pre-rendered demo

10. Licensing and ethics

- **Ultralytics YOLO** is open source under AGPL-3.0, with a separate enterprise licence for commercial use. That is fine for learning and open-source work. Read the terms before building anything commercial on it.
- **Datasets** have their own licences (many on Roboflow Universe are CC BY 4.0, but check each page). Credit the authors in your README.
- **Match footage is copyrighted.** Don't publish clips you don't have the rights to. Use your own recordings, openly licensed clips, or publish only results and short, clearly attributed excerpts you are comfortable defending.
- **People in footage.** Your system detects and tracks individuals. Don't use it to identify named people, and keep it to sport analysis.
- **Be honest about limits.** Your possession figure is an estimate, and your ball detector will miss. Say so.

11. Stretch ideas beyond the plan

- Combine a second dataset after harmonising class names, and report whether it helped.
- Detect passes and shots from ball trajectory changes.
- Add a pass-network graph for each team.
- Export to ONNX or TensorRT and compare speed.
- Compare YOLO26 against the previous generation, YOLO11, on your test set.

12. Final checklist

- [ ] Baseline and fine-tuned results compared on a held-out clip you labelled yourself
- [ ] Experiment log with at least four runs
- [ ] Failure gallery and a limitations section
- [ ] Tracking with IDs; ID-switch count recorded
- [ ] Ball path cleaned, with gaps left honest
- [ ] Teams assigned; accuracy checked on 50 crops
- [ ] Possession shown and labelled as an estimate
- [ ] `run.py` works from the command line
- [ ] Gradio demo deployed

- [ ] README with demo GIF, results table and licences
- [ ] Weights, dataset zip and notebooks backed up